

MARTIN Company

Sistema de Gestión de Producción de Uniformes Médicos

Informe técnico: Antes vs Ahora

Evolución del sistema: del modelo plano a un esquema relacional en PostgreSQL (Supabase), con API REST/RPC, flujo de producción completo, roles, calidad y capa de demostración mock.

Proyecto: tesis-cami-salesiana

Fecha: Julio 2026

Alcance: arquitectura, flujo operativo, base de datos y UX

1. Resumen ejecutivo

El sistema MARTIN Company pasó de un inventario/pedidos con tablas planas y estados básicos a una arquitectura v2 centrada en PostgreSQL hospedado en Supabase. La aplicación web (HTML/CSS/JS) consume la API de Supabase (PostgREST + RPC) para persistir y orquestar el flujo: pedido → verificación de stock → compra (si falta materia) → orden de producción → avances → control de calidad (10 criterios) → entrega.

Además se incorporó un modo mock en JavaScript para demos sin depender del SQL, mensajes toast en lugar de alertas nativas, y botones de acción con diseño por contexto.

Objetivos del rediseño

- Modelar el negocio real (variantes producto+color+talla, recetas, maquiladoras).
- Garantizar integridad referencial y reglas de negocio en la base de datos (funciones SQL).
- Separar roles: supervisora vs maquiladora.
- Cubrir el ciclo completo hasta calidad y entrega, con KPIs.
- Facilitar pruebas y presentación (seed SQL + mock JS).

2. Qué se analizó

Antes de rediseñar se revisaron estos puntos del sistema anterior y del dominio:

- Tablas planas (producto/talla/color como texto libre) que generaban duplicados.
- Estados de pedido limitados (pendiente / en_proceso / completado), insuficientes.
- Ausencia de órdenes de producción, avances, alertas de retraso e inspección de calidad.
- Inventario sin receta (metros por talla) ni descuento automático de materia/suministros.
- Sin perfiles/roles: cualquier usuario veía todo el sistema.
- Documentación de negocio (Excel de stock/recetas y Word de criterios) no reflejada en el modelo.
- Dependencia de alert() nativo y botones sin semántica visual en tablas.

Fuentes de análisis

- Esquema SQL v1: productos_terminados, productos_proceso, stock_disponible, pedidos planos.
- Módulos JS previos: inventario, pedidos, reportes con CRUD básico vía Supabase.
- Requisitos de flujo: stock insuficiente → compra; stock OK → OP; avance 100% → calidad; aprobado → entrega.
- Criterios de calidad (10 parámetros con criticidad Alta/Media/Baja).

3. Cambios: antes vs ahora

3.1 Comparación general

Aspecto	ANTES (v1)	AHORA (v2)
Modelo de datos	Tablas planas, textos libres	Esquema relacional normalizado
Producto	Texto producto+talla+color	producto_variantes (FK)
Pedidos	1 fila = 1 producto	pedido + pedido_items (N ítems)
Estados pedido	4 estados básicos	11 estados del flujo real
Producción	Sin OP formal	órdenes + avances + alertas
Calidad	No existía	10 criterios + inspección + RPC
Stock materia	Cantidad simple	Receta por talla + descuento en OP
Roles	Sin login/roles	perfiles: supervisora / maquiladora
Reglas negocio	Solo en JS	Funciones PostgreSQL (RPC)
KPIs	Conteos básicos	vista_kpis + dashboard
Demo/pruebas	Datos manuales	seed SQL + mock JS + login demo
UX mensajes	alert() del navegador	Toasts éxito/aviso/error
Botones tabla	Estilo genérico	Colores por acción (OP, compra...)

3.2 Tablas eliminadas / reemplazadas

- ANTES: productos_terminados, productos_proceso, stock_disponible (duplicaban el mismo concepto).
- AHORA: producto_variantes.stock_terminado + ordenes_produccion + flujo de estados.

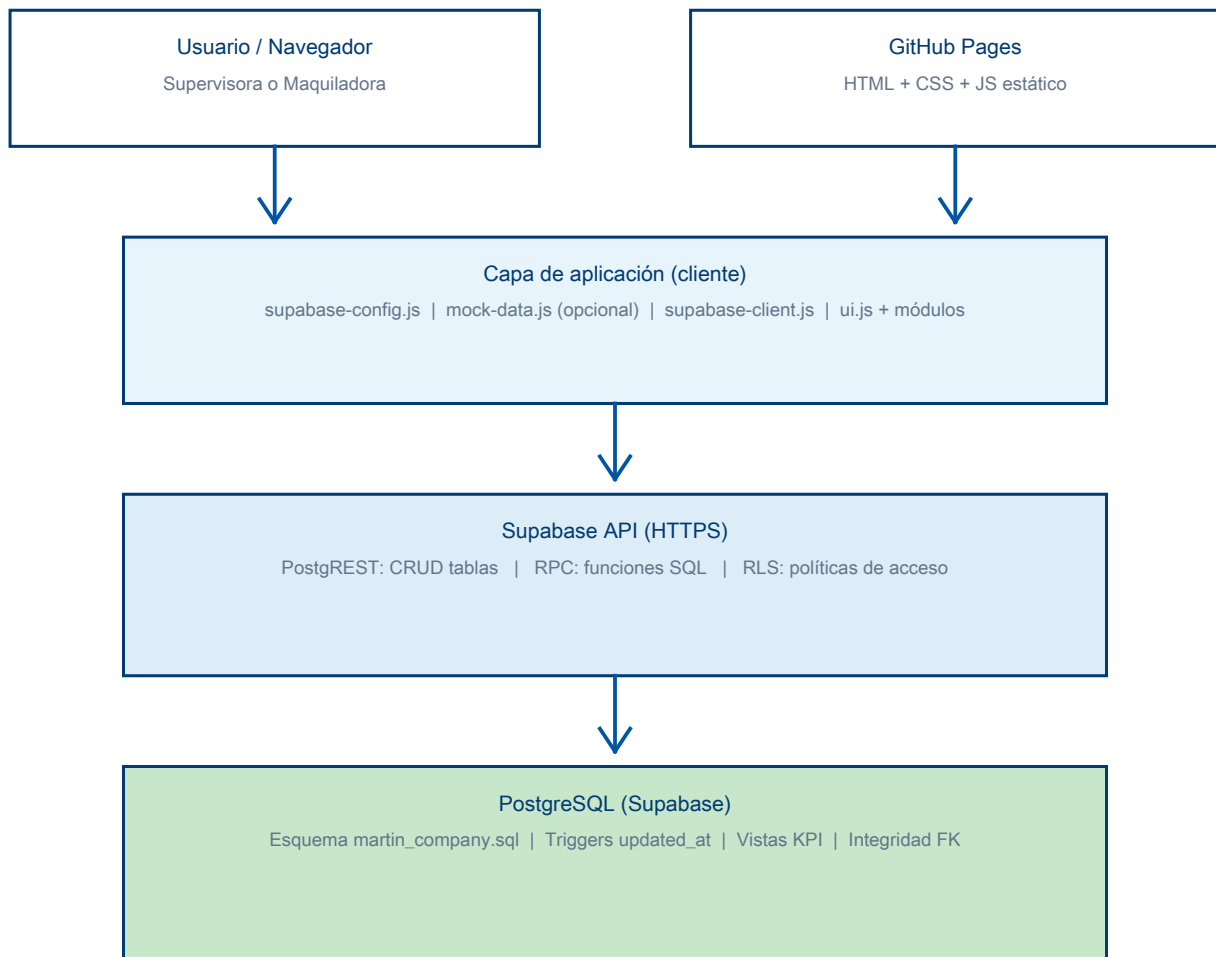
3.3 Tablas y objetos nuevos (v2)

- perfiles, clientes, proveedores, productos, colores, tallas, producto_variantes
- materia_prima, suministros, receta_materia, receta_suministros
- pedidos, pedido_items, solicitudes_compra, solicitud_compra_items, ingresos_materia
- ordenes_produccion, consumo_produccion, avances_produccion, alertas_produccion
- criterios_calidad, inspecciones_calidad, inspeccion_items, entregas
- Vistas: vista_kpis, vista_pedidos_proceso
- RPC: verificar_stock_pedido, generar_orden_produccion, registrar_avance_produccion, aprobar_inspeccion_calidad, recibir_materia_compra

4. Arquitectura actual

PostgreSQL es el motor de datos. Supabase lo expone como servicio: API REST automática (PostgREST) sobre las tablas, y llamadas RPC a funciones SQL. El frontend estático (GitHub Pages) usa el SDK `@supabase/supabase-js`.

4.1 Diagrama de arquitectura

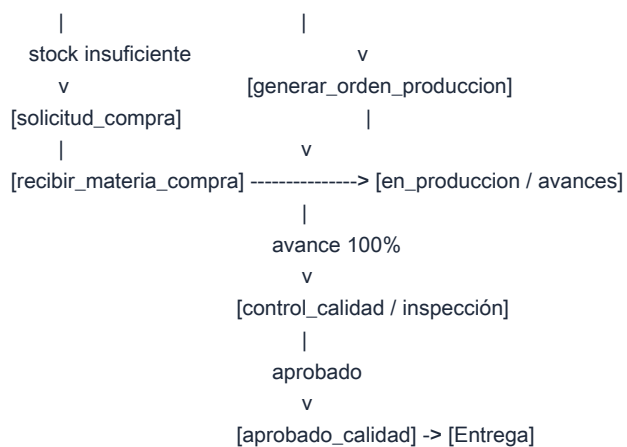


Proceso de una operación típica (cómo se usa la API)

- 1. La UI llama `obtenerRegistros / insertarRegistro / llamarRPC` (`supabase-client.js`).
- 2. El SDK envía HTTP a `https://<proyecto>.supabase.co/rest/v1/... o /rpc/...`
- 3. PostgREST traduce a SQL sobre PostgreSQL (`SELECT/INSERT/UPDATE` o `CALL` función).
- 4. Las funciones SQL aplican reglas (stock, descuentos, estados) de forma atómica.
- 5. La respuesta JSON actualiza tablas/KPIs; `ui.js` muestra toast de éxito o error.
- 6. Si `USE MOCK DATA=true`, `mock-data.js` simula la misma API en `localStorage` (demo sin red).

4.2 Diagrama del flujo de negocio





Las transiciones críticas viven en PostgreSQL (RPC), no solo en el navegador. Así se evita inconsistencia si hay varios usuarios.

5. Mejoras del flujo operativo

5.1 Qué mejoró en el proceso

Bifurcación por stock

Al registrar un pedido se calcula la materia según receta (metros S/M/L). Si falta, se crea solicitud de compra; si alcanza, queda listo para OP.

Orden de producción

Al generar OP se descuenta materia y suministros, se asigna maquiladora y fecha planeada.

Avances y retrasos

La maquiladora registra %; si se pasa la fecha con avance < 100% se crea alerta.

Control de calidad

10 criterios del documento Word; si falla un criterio no se aprueba ni se suma stock terminado.

Entrega

Solo pedidos en aprobado_calidad pueden entregarse; el dashboard refleja entregas e inspecciones.

Roles

Supervisora: inventario, pedidos, calidad, reportes. Maquiladora: producción de sus órdenes.

5.2 Capas de prueba

- seed_flujo_mock.sql: 7 pedidos (PED-001...007) en estados distintos para recorrer el flujo en Supabase.
- js/mock-data.js: misma narrativa sin SQL, con restauración desde el login.

6. Por qué ahora es mejor

Criterio	Impacto	Beneficio
Integridad de datos	FK + UNIQUE variantes	Menos duplicados y errores de tipeo
Reglas en el servidor	RPC PostgreSQL	Lógica confiable aunque falle el JS
Trazabilidad	Avances, consumo, inspecciones	Auditoría del pedido extremo a extremo
Escalabilidad	API REST + Postgres	Varios usuarios / dispositivos a la vez
Mantenibilidad	Módulos por dominio	Cambios localizados (pedidos, calidad...)
Demostración	Mock + seed	Presentación de tesis sin fricción
UX	Toasts + botones semánticos	Feedback claro sin alertas bloqueantes
Despliegue	GitHub Pages + Supabase	Frontend gratis, backend gestionado

Conclusión

El sistema dejó de ser un CRUD de tablas sueltas y pasó a ser un flujo de producción soportado por PostgreSQL en Supabase. La API REST/RPC conecta la UI con reglas de negocio centralizadas. Eso reduce inconsistencias, refleja el proceso real de MARTIN Company y facilita la evaluación académica y operativa del producto.

7. Entregables técnicos clave

- supabase/martin_company.sql — esquema v2 completo
- supabase/seed_flujo_mock.sql — datos de flujo de prueba
- js/supabase-client.js — cliente API (o mock)
- js/mock-data.js — demo offline
- js/ui.js — toasts y confirmaciones
- Módulos: pedidos, producción, calidad, inventario, reportes, login con roles

Documento generado a partir del estado del repositorio tesis-cami-salesiana (rediseño v2 + mejoras UX).